

K-means Clustering

This notebook serves as a reference for the code used in the lecture. Please view the video lecture for a full explanation of the methods used here.

Method Used

K Means Clustering is an unsupervised learning algorithm that tries to cluster data based on their similarity. Unsupervised learning means that there is no outcome to be predicted, and the algorithm just tries to find patterns in the data. In k means clustering, we have to specify the number of clusters we want the data to be grouped into. The algorithm randomly assigns each observation to a cluster, and finds the centroid of each cluster. Then, the algorithm iterates through two steps:

Reassign data points to the cluster whose centroid is closest. Calculate new centroid of each cluster. These two steps are repeated till the within cluster variation cannot be reduced any further. The within cluster variation is calculated as the sum of the euclidean distance between the data points and their respective cluster centroids.

Get the Data

We will use the [iris](#) data set. The Iris flower data set or Fisher's Iris data set is a multivariate data set introduced by Ronald Fisher in his 1936 paper *The use of multiple measurements in taxonomic problems as an example of linear discriminant analysis*.

The data set consists of 50 samples from each of three species of Iris (Iris setosa, Iris virginica and Iris versicolor). Four features were measured from each sample: the length and the width of the sepals and petals, in centimetres.

In [1]:

```
library(datasets)
```

In [2]:

```
head(iris)
```

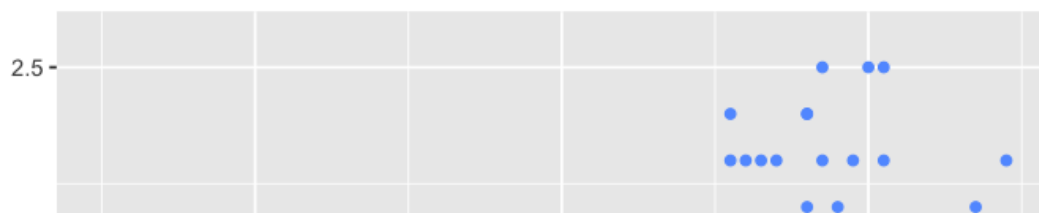
Out[2]:

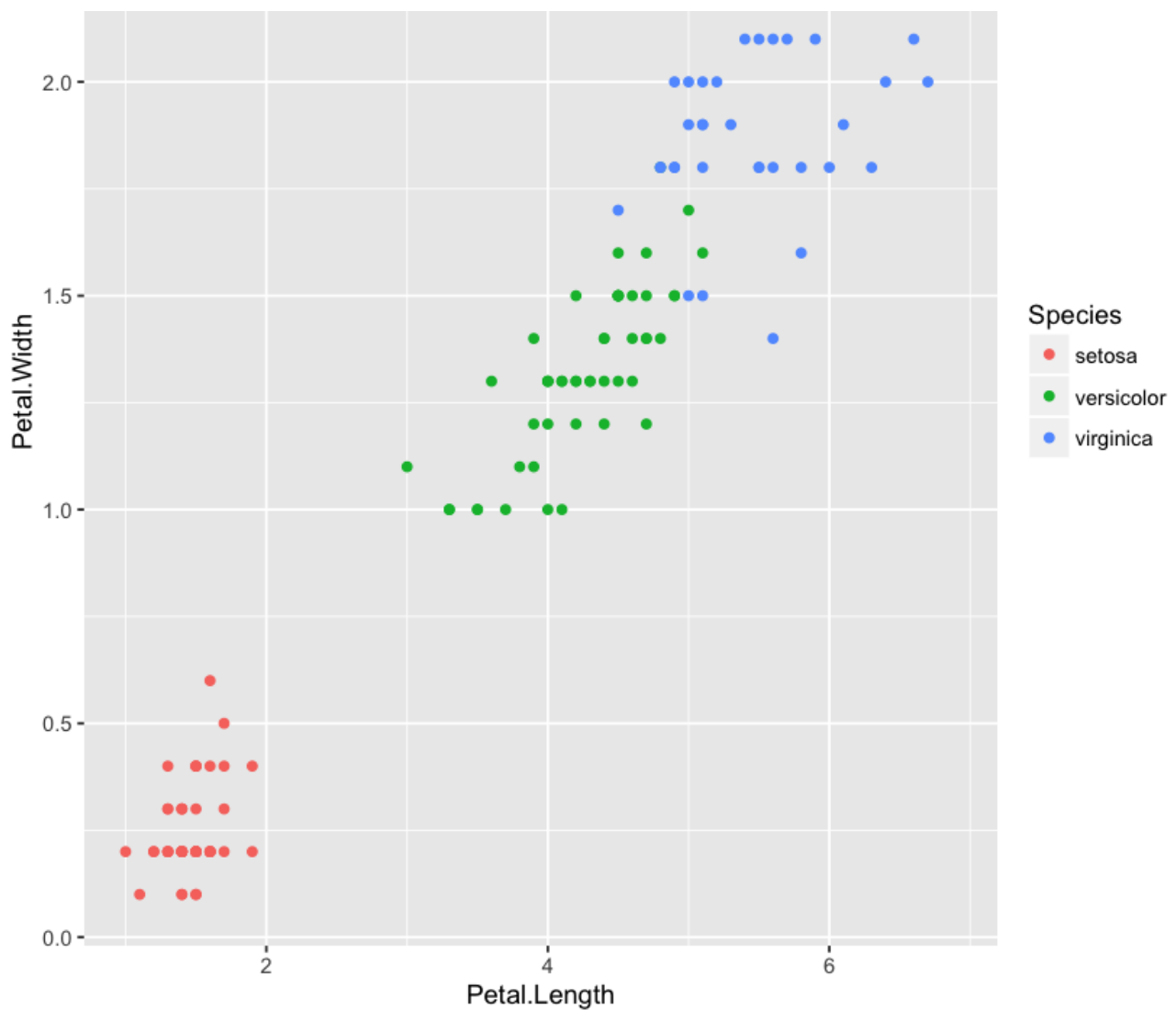
	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

EDA

In [3]:

```
library(ggplot2)
ggplot(iris, aes(Petal.Length, Petal.Width, color = Species)) + geom_point()
```





Clustering

Now let's attempt to use the K-means algorithm to cluster the data. Remember that this is an unsupervised learning algorithm, meaning we won't give it any information on the correct labels:

In [9]:

```
set.seed(101)
```

In [10]:

```
help(kmeans)
```

Out[10]:

kmeans {stats}	R Documentation
----------------	-----------------

K-Means Clustering

Description

Perform k-means clustering on a data matrix.

Usage

```
kmeans(x, centers, iter.max = 10, nstart = 1,  
       algorithm = c("Hartigan-Wong", "Lloyd", "Forgy",  
                     "MacQueen"), trace=FALSE)
```

```

      macqueen , , FALSE)
## S3 method for class 'kmeans'
fitted(object, method = c("centers", "classes"), ...)

```

Arguments

<code>x</code>	numeric matrix of data, or an object that can be coerced to such a matrix (such as a numeric vector or a data frame with all numeric columns).
<code>centers</code>	either the number of clusters, say k , or a set of initial (distinct) cluster centres. If a number, a random set of (distinct) rows in <code>x</code> is chosen as the initial centres.
<code>iter.max</code>	the maximum number of iterations allowed.
<code>nstart</code>	if <code>centers</code> is a number, how many random sets should be chosen?
<code>algorithm</code>	character: may be abbreviated. Note that "Lloyd" and "Forgy" are alternative names for one algorithm.
<code>object</code>	an R object of class "kmeans", typically the result <code>ob</code> of <code>ob <- kmeans(...)</code> .
<code>method</code>	character: may be abbreviated. "centers" causes <code>fitted</code> to return cluster centers (one for each input point) and "classes" causes <code>fitted</code> to return a vector of class assignments.
<code>trace</code>	logical or integer number, currently only used in the default method ("Hartigan-Wong"): if positive (or true), tracing information on the progress of the algorithm is produced. Higher values may produce more tracing information.
<code>...</code>	not used.

Details

The data given by `x` are clustered by the k -means method, which aims to partition the points into k groups such that the sum of squares from points to the assigned cluster centres is minimized. At the minimum, all cluster centres are at the mean of their Voronoi sets (the set of data points which are nearest to the cluster centre).

The algorithm of Hartigan and Wong (1979) is used by default. Note that some authors use k -means to refer to a specific algorithm rather than the general method: most commonly the algorithm given by MacQueen (1967) but sometimes that given by Lloyd (1957) and Forgy (1965). The Hartigan–Wong algorithm generally does a better job than either of those, but trying several random starts (`nstart` > 1) is often recommended. In rare cases, when some of the points (rows of `x`) are extremely close, the algorithm may not converge in the “Quick-Transfer” stage, signalling a warning (and returning `ifault` = 4). Slight rounding of the data may be advisable in that case.

For ease of programmatic exploration, $k=1$ is allowed, notably returning the center and `withinss`.

Except for the Lloyd–Forgy method, k clusters will always be returned if a number is specified. If an initial matrix of centres is supplied, it is possible that no point will be closest to one or more centres, which is currently an error for the Hartigan–Wong method.

Value

`kmeans` returns an object of class "kmeans" which has a `print` and a `fitted` method. It is a list with at least the following components:

<code>cluster</code>	A vector of integers (from 1: k) indicating the cluster to which each point is allocated.
<code>centers</code>	A matrix of cluster centres.
<code>totss</code>	The total sum of squares.
<code>withinss</code>	Vector of within-cluster sum of squares, one component per cluster.
<code>tot.withinss</code>	Total within-cluster sum of squares, i.e. <code>sum(withinss)</code> .
<code>betweenss</code>	The between-cluster sum of squares, i.e. <code>totss-tot.withinss</code> .
<code>size</code>	The number of points in each cluster.
<code>iter</code>	The number of (outer) iterations.
<code>ifault</code>	integer: indicator of a possible algorithm problem – for experts.

References

Forgy, E. W. (1965) Cluster analysis of multivariate data: efficiency vs interpretability of classifications. *Biometrics* **21**, 768–769.

Hartigan, J. A. and Wong, M. A. (1979). A K-means clustering algorithm. *Applied Statistics* **28**, 100–108.

Lloyd, S. P. (1957, 1982) Least squares quantization in PCM. Technical Note, Bell Laboratories. Published in 1982 in *IEEE Transactions on Information Theory* **28**, 128–137.

MacQueen, J. (1967) Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, eds L. M. Le Cam & J. Neyman, **1**, pp. 281–297. Berkeley, CA: University of California Press.

Examples

require(graphics)

```
# a 2-dimensional example
x <- rbind(matrix(rnorm(100, sd = 0.3), ncol = 2),
           matrix(rnorm(100, mean = 1, sd = 0.3), ncol = 2))
colnames(x) <- c("x", "y")
(cl <- kmeans(x, 2))
plot(x, col = cl$cluster)
points(cl$centers, col = 1:2, pch = 8, cex = 2)

# sum of squares
ss <- function(x) sum(scale(x, scale = FALSE)^2)

## cluster centers "fitted" to each obs.:
fitted.x <- fitted(cl); head(fitted.x)
resid.x <- x - fitted(cl)

## Equalities : -----
cbind(cl[c("betweenss", "tot.withinss", "totss")], # the same two columns
      c(ss(fitted.x), ss(resid.x), ss(x)))
stopifnot(all.equal(cl$ totss, ss(x)),
          all.equal(cl$ tot.withinss, ss(resid.x)),
          ## these three are the same:
          all.equal(cl$ betweenss, ss(fitted.x)),
          all.equal(cl$ betweenss, cl$totss - cl$tot.withinss),
          ## and hence also
          all.equal(ss(x), ss(fitted.x) + ss(resid.x))
)

kmeans(x,1)$withinss # trivial one-cluster, (its W.SS == ss(x))

## random starts do help here with too many clusters
## (and are often recommended anyway!):
(cl <- kmeans(x, 5, nstart = 25))
plot(x, col = cl$cluster)
points(cl$centers, col = 1:5, pch = 8)
```

[Package *stats* version 3.2.2]

In [11]:

```
# Luckily we already know how many clusters to expect
irisCluster <- kmeans(iris[, 1:4], 3, nstart = 20)
irisCluster
```

Out[11]:

K-means clustering with 3 clusters of sizes 62, 50, 38

Cluster means:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
1	5.901613	2.748387	4.393548	1.433871
2	5.006000	3.428000	1.462000	0.246000

3	6.850000	3.073684	5.742105	2.071053
---	----------	----------	----------	----------

Clustering vector:

[illegible]

Within cluster sum of squares by cluster:

```
[1] 39.82097 15.15100 23.87947
      (between_SS / total_SS =  88.4 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

In [13]:

```
table(irisCluster$cluster, iris$Species)
```

Out[13]:

	setosa	versicolor	virginica
1	0	48	14
2	50	0	0
3	0	2	36

We can also grab additional information about the clusters.

In [14]:

irisCluster

Out[14]:

	Length	Class	Mode
cluster	150	-none-	numeric
centers	12	-none-	numeric
totss	1	-none-	numeric
withinss	3	-none-	numeric
tot.withinss	1	-none-	numeric
betweenss	1	-none-	numeric
size	3	-none-	numeric
iter	1	-none-	numeric
ifault	1	-none-	numeric

Looks like we correctly grouped the setosa, with some crossover with versicolor and virginica.

Cluster Visualizations

We can plot the clusters out:

In [20]:

```
library(cluster)
```

In [36]:

```
help(clusplot)
```

Out[36]:

Bivariate Cluster Plot (of a Partitioning Object)

Description

Draws a 2-dimensional “clusplot” (clustering plot) on the current graphics device. The generic function has a default and a partition method.

Usage

```
clusplot(x, ...)
```

```
## S3 method for class 'partition'  
clusplot(x, main = NULL, dist = NULL, ...)
```

Arguments

x	an R object, here, specifically an object of class "partition", e.g. created by one of the functions <code>pam</code> , <code>clara</code> , or <code>fanny</code> .
main	title for the plot; when <code>NULL</code> (by default), a title is constructed, using <code>x\$call</code> .
dist	when <code>x</code> does not have a <code>diss</code> nor a data component, e.g., for <code>pam(dist(*), keep.diss=FALSE)</code> , <code>dist</code> must specify the dissimilarity for the <code>clusplot</code> .
...	optional arguments passed to methods, notably the <code>clusplot.default</code> method (except for the <code>diss</code> one) may also be supplied to this function. Many graphical parameters (see <code>par</code>) may also be supplied as arguments here.

Details

The `clusplot.partition()` method relies on `clusplot.default`.

If the clustering algorithms `pam`, `fanny` and `clara` are applied to a data matrix of observations-by-variables then a `clusplot` of the resulting clustering can always be drawn. When the data matrix contains missing values and the clustering is performed with `pam` or `fanny`, the dissimilarity matrix will be given as input to `clusplot`. When the clustering algorithm `clara` was applied to a data matrix with NAs then `clusplot` will replace the missing values as described in `clusplot.default`, because a dissimilarity matrix is not available.

Value

For the `partition` (and `default`) method: An invisible list with components `Distances` and `Shading`, as for `clusplot.default`, see there.

Side Effects

a 2-dimensional `clusplot` is created on the current graphics device.

See Also

`clusplot.default` for references; `partition.object`, `pam`, `pam.object`, `clara`, `clara.object`, `fanny`, `fanny.object`, `par`.

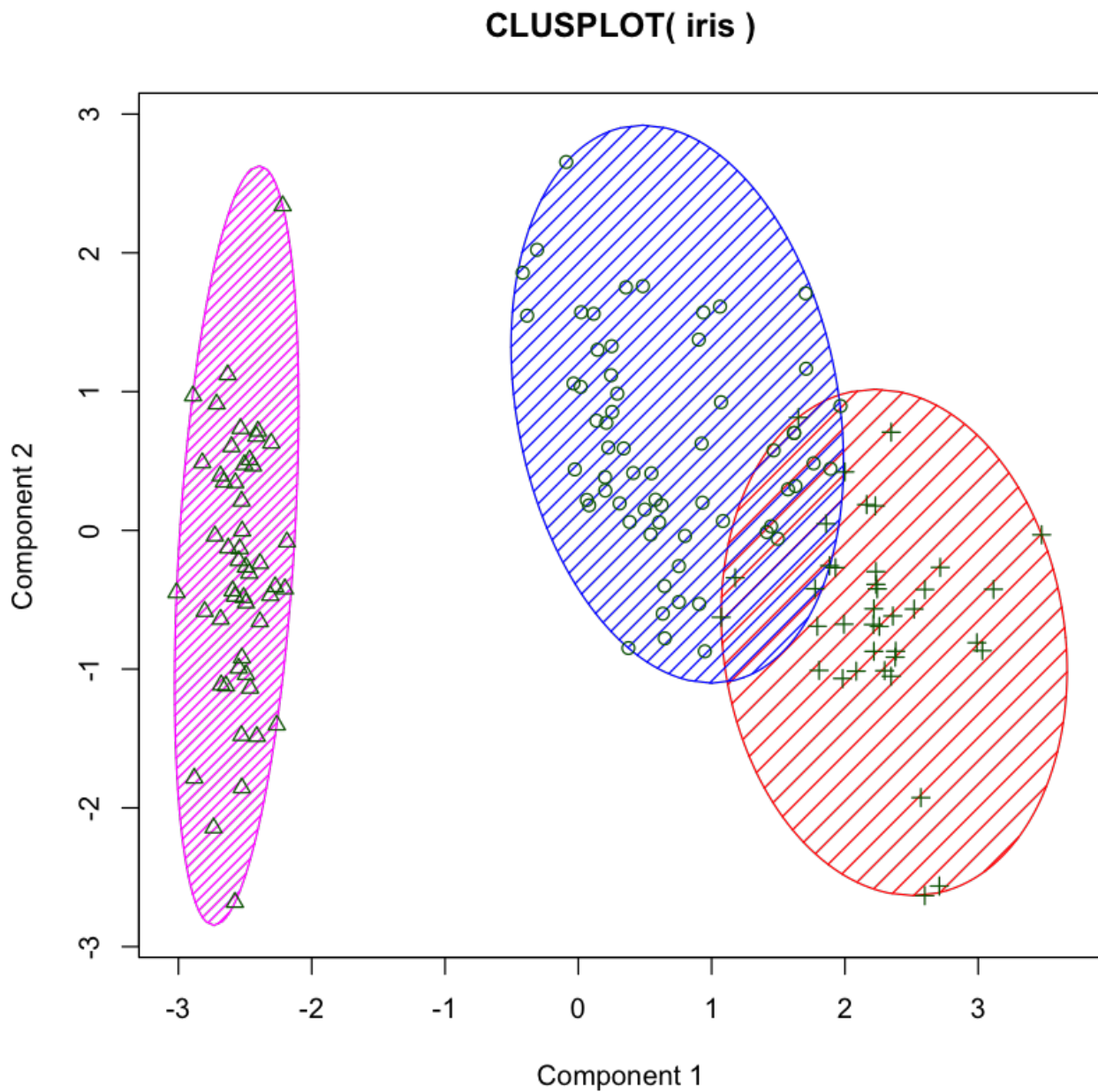
Examples

```
## For more, see ?clusplot.default  
  
## generate 25 objects, divided into 2 clusters.  
x <- rbind(cbind(rnorm(10,0,0.5), rnorm(10,0,0.5)),  
           cbind(rnorm(15,5,0.5), rnorm(15,5,0.5)))  
clusplot(pam(x, 2))  
## add noise, and try again :  
x4 <- cbind(x, rnorm(25), rnorm(25))  
clusplot(pam(x4, 2))
```

Here we can plot two components and the cluster regions versus the actual real data labels:

In [35]:

```
library(cluster)
clusplot(iris, irisCluster$cluster, color=TRUE, shade=TRUE, labels=0, lines=0, )
```



These two components explain 95.02 % of the point variability.

Great Job!